

广电大数据用户画像分析

项目类型：离线用户画像系统建设

技术栈：Hadoop / Hive / Spark / Python

数据规模：5张业务表，百万级用户数据

产出：8维度用户标签宽表 + SVM挽留预测模型

目录

一、数据存储与初步认识	2
1.1 数据说明	2
1.2 数据探索	3
二、预处理与特征构建	4
2.1 异常数据	4
2.2 主要业务数据探索	6
2.3 标签阈值探索	10
2.4 数据预处理	17
三、SVM 模型构建与用户画像生成	20
3.1 构建特征列和标签列数据	20
3.2 SVM 预测用户是否挽留	23
3.3 构建用户画像	24
四、设计体会	32
4.1 理解与收获	32
4.2 遇到的主要问题及解决方法	33
4.3 改进方面	33

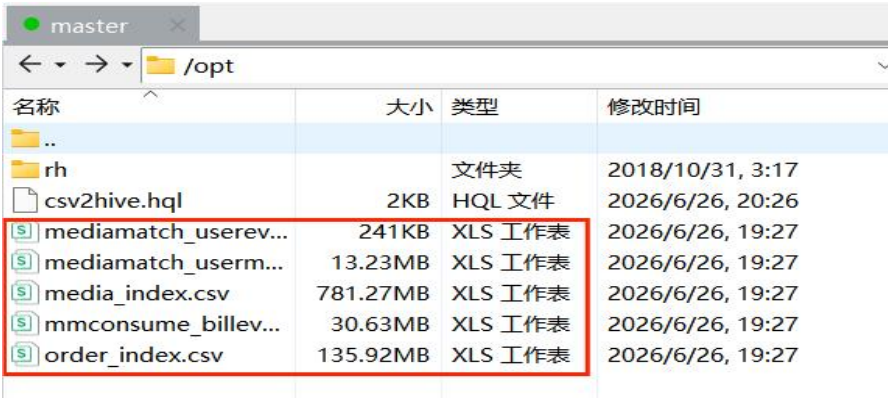
一、数据存储与初步认识

1.1 数据说明

业务数据表有用户基本信息表、账单信息表、订单信息表、用户状态信息变更表以及用户收视行为信息表。

表名	作用	关键字段
mediamatch_usermsg 用户信息主表	记录用户的基本属性与当前状态	phone_no, sm_name, run_name, open_time
mediamatch_userevent 用户状态变更表	记录用户状态的历史流转	phone_no, run_name, run_time, owner_name
mmconsume_billevts 账单信息表	反映用户的消费能力	phone_no, fee_code, should_pay, year_month
order_index 订单信息表	记录用户的产品偏好；	phone_no, sm_name, offername, effdate, cost
media_index 收视行为表	量化用户的内容偏好	phone_no, duration, origin_time, vod_cat_tags

（一）将 5 张业务表文件上传到 Linux 操作系统的/opt 目录下，使用“sed”命令删除表中的首行字段名。



```
[root@master ~]# ls -lh /opt/*.csv
-rw-r--r--. 1 root root 782M Jun 25 07:05 /opt/media_index.csv
-rw-r--r--. 1 root root 242K Jun 25 07:06 /opt/mediamatch_userevent.csv
-rw-r--r--. 1 root root 14M Jun 25 07:06 /opt/mediamatch_usermsg.csv
-rw-r--r--. 1 root root 31M Jun 25 07:06 /opt/mmconsume_billevents.csv
-rw-r--r--. 1 root root 136M Jun 25 07:07 /opt/order_index.csv

[root@master opt]# sed -i.bak '1d' /opt/mediamatch_usermsg.csv
[root@master opt]# sed -i.bak '1d' /opt/mediamatch_userevent.csv
[root@master opt]# sed -i.bak '1d' /opt/mmconsume_billevents.csv
[root@master opt]# sed -i.bak '1d' /opt/order_index.csv
[root@master opt]# sed -i.bak '1d' /opt/media_index.csv
```

（二）启动 Hadoop 集群、Hive 的 MetaStore 服务，在 Shell 中执行 HiveQL 命令，即可完成 5 张业务表的创建与数据导入。

```
Loading data to table user_profile.mediainmatch_usermsg
OK
Time taken: 9.74 seconds
OK
Time taken: 0.307 seconds
Loading data to table user_profile.mediainmatch_userevent
OK
Time taken: 0.701 seconds
OK
Time taken: 0.372 seconds
Loading data to table user_profile.mmconsume_billevents
OK
Time taken: 1.034 seconds
OK
Time taken: 0.282 seconds
Loading data to table user_profile.order_index
OK
Time taken: 2.109 seconds
OK
Time taken: 0.18 seconds
Loading data to table user_profile.media_index_raw
OK
Time taken: 9.086 seconds
```

1.2 数据探索

（一）在 Linux 环境下启动 Hadoop 与 Hive 服务，编写简单查询，验证数据导入是否成功，并统计各表的记录总数（变量名称自定义）。

（二）对这 5 张业务数据表进行总体的分析概述，了解其记录数、用

户数、观看时长的均值、最值、标准差和用户月均观看时长。

```
scala> println("=== 各表记录数 ===")
=== 各表记录数 ===

scala> val cnt1 = spark.table("mediamatch_usermsg").count()
cnt1: Long = 100000

scala> val cnt2 = spark.table("mediamatch_userevent").count()
cnt2: Long = 3000

scala> val cnt3 = spark.table("mmconsume_billevents").count()
cnt3: Long = 439158

scala> val cnt4 = spark.table("order_index").count()
cnt4: Long = 608514
```

```
scala> val cnt = spark.table("user_profile.media_index").count()
cnt: Long = 4754442

scala> println(s"media_index 表共有 $cnt 条记录")
media_index 表共有 4754442 条记录
```

```
scala> println(s"用户信息表: $cnt1")
用户信息表: 100000

scala> println(s"状态变更表: $cnt2")
状态变更表: 3000

scala> println(s"账单信息表: $cnt3")
账单信息表: 439158

scala> println(s"订单信息表: $cnt4")
订单信息表: 608514
```

```
scala> println(f"平均观看时长: ${stats.getDouble(0)}%.2f 小时")
平均观看时长: 0.31 小时

scala> println(f"最大观看时长: ${stats.getDouble(1)}%.2f 小时")
最大观看时长: 5.00 小时

scala> println(f"最小观看时长: ${stats.getDouble(2)}%.2f 小时")
最小观看时长: 0.00 小时

scala> println(f"标准差: ${stats.getDouble(3)}%.2f 小时")
标准差: 0.40 小时

scala> █
```

二、预处理与特征构建

2.1 异常数据

主要是查看用户信息表是否有重复记录的用户、是否存在特殊线路的用户以及政企用户。

(一) 检查用户基本信息表中是否存在重复记录的用户。

```
scala> println("=== 检查重复用户 ===")
=== 检查重复用户 ===

scala> val dupUsers = spark.sql("""
  SELECT phone_no, COUNT(*) as cnt
  FROM mediamatch_usermsg
  GROUP BY phone_no
  HAVING COUNT(*) > 1
  """)
dupUsers: org.apache.spark.sql.DataFrame = [phone_no: string, cnt: bigint]

scala> val dupCount = dupUsers.count()
dupCount: Long = 0

scala> println(s"重复用户数: $dupCount")
重复用户数: 0
```

(二) 查看特殊线路的用户以及政企用户。

```
scala> println("=== 特殊线路用户统计（各表分布） ===")
=== 特殊线路用户统计（各表分布） ===

scala> val specialCodes = Seq("02", "09", "10")
specialCodes: Seq[String] = List(02, 09, 10)

scala>

scala> // 用户信息表

scala> val cnt1 = spark.sql(s"SELECT COUNT(*) FROM mediamatch_usermsg WHERE owner_code IN (${specialCodes.map(c => s"'$c'").mkString(",")})").collect()(0).getLong(0)
cnt1: Long = 124

scala> println(s"用户信息表中特殊线路用户数: $cnt1")
用户信息表中特殊线路用户数: 124
```

```
scala> // 订单信息表

scala> val cnt3 = spark.sql(s"SELECT COUNT(*) FROM order_index WHERE owner_code IN (${specialCodes.map(c => s"'$c'").mkString(",")})").collect()(0).getLong(0)
cnt3: Long = 146
```

```
scala> // 账单信息表

scala> val cnt2 = spark.sql(s"SELECT COUNT(*) FROM mmconsume_billevents WHERE owner_code IN (${specialCodes.map(c => s"'$c'").mkString(",")})").collect()(0).getLong(0)
cnt2: Long = 155

scala> println(s"账单信息表中特殊线路用户数: $cnt2")
账单信息表中特殊线路用户数: 155
```

```
scala> // 收视行为表（用临时视图）

scala> val cnt4 = spark.sql(s"SELECT COUNT(*) FROM media_index_temp WHERE owner_code IN (${specialCodes.map(c => s"'$c'").mkString(",")})").collect()(0).getLong(0)
cnt4: Long = 2480

scala> println(s"收视行为表中特殊线路用户数: $cnt4")
收视行为表中特殊线路用户数: 2480
```


（三）说明：根据业务规则（客户等级为 02、09、10 为特殊线路；政企用户的标识是 owner_name 的值为“EA 级”“EB 级”“EC 级”“ED 级”或“EE 级”），统计这些异常用户在各表中的分布情况，并在后续处理中予以剔除。

```
scala> println("=== 政企用户统计（各表分布）===")
=== 政企用户统计（各表分布）===

scala> val govNames = Seq("EA级", "EB级", "EC级", "ED级", "EE级")
govNames: Seq[String] = List(EA级, EB级, EC级, ED级, EE级)

scala> val nameList = govNames.map(n => s"'$n'").mkString(",")
nameList: String = 'EA级','EB级','EC级','ED级','EE级'

scala> val cnt1_gov = spark.sql(s"SELECT COUNT(*) FROM mediamatch_usermsg WHERE owner_name IN ($nameList)").collect()(0).getLong(0)
cnt1_gov: Long = 6368

scala> println(s"用户信息表中政企用户数: $cnt1_gov")
用户信息表中政企用户数: 6368

scala> val cnt2_gov = spark.sql(s"SELECT COUNT(*) FROM mmconsume_billevents WHERE owner_name I
N ($nameList)").collect()(0).getLong(0)
cnt2_gov: Long = 17473

scala> println(s"账单信息表中政企用户数: $cnt2_gov")
账单信息表中政企用户数: 17473
```

```
scala> val cnt3_gov = spark.sql(s"SELECT COUNT(*) FROM order_index WHERE owner_name IN ($nameL
ist)").collect()(0).getLong(0)
cnt3_gov: Long = 28958

scala> println(s"订单信息表中政企用户数: $cnt3_gov")
订单信息表中政企用户数: 28958

scala> val cnt4_gov = spark.sql(s"SELECT COUNT(*) FROM media_index_temp WHERE owner_name IN ($
nameList)").collect()(0).getLong(0)
cnt4_gov: Long = 76933

scala> println(s"收视行为表中政企用户数: $cnt4_gov")
收视行为表中政企用户数: 76933
```

2.2 主要业务数据探索

广电集团的对主要业务数据需求如下。

- （1） 业务品牌是互动电视、甜果电视、数字电视、珠江宽频；
- （2） 主要状态用户需要是正常、欠费暂停、主动暂停和主动销户；
- （3） 有效的收视数据。

本小节将对数据表中的品牌类型、用户状态、收视时长进行探索。

(一) 统计用户基本信息表中 `sm_name` 的所有类型及其每种类型的用户数。
业务品牌仅保留：互动电视、甜果电视、数字电视、珠江宽频。

```
scala> // 1. 统计所有品牌类型及用户数

scala> println("所有品牌类型及用户数：")
所有品牌类型及用户数：

scala> spark.sql("""
    SELECT sm_name, COUNT(*) as cnt
    FROM mediamatch_usermsg
    GROUP BY sm_name
    ORDER BY cnt DESC
    """).show(20, false)
+-----+-----+
|sm_name|cnt|
+-----+-----+
|模拟有线电视|49451|
|数字电视|30131|
|互动电视|8341|
|珠江宽频|6850|
|甜果电视|5227|
+-----+-----+
```

```
scala> // 2. 统计四种核心品牌的用户数

scala> println("四种核心品牌用户数：")
四种核心品牌用户数：

scala> spark.sql("""
    SELECT sm_name, COUNT(*) as cnt
    FROM mediamatch_usermsg
    WHERE sm_name IN ('互动电视', '甜果电视', '数字电视', '珠江宽频')
    GROUP BY sm_name
    """).show()
+-----+-----+
|sm_name|cnt|
+-----+-----+
|互动电视|8341|
|数字电视|30131|
|珠江宽频|6850|
|甜果电视|5227|
+-----+-----+
```

(二) 对用户状态名称利用 `groupBy`、`count` 进行分组统计，状态名称包含空值一共有 8 种类型，其中只保留正常、欠费暂停、主动暂停和主动销户的用户，其余的状态名称不需要分析处理。

```
scala> // 1. 统计所有状态类型及数量
```

```
scala> println("所有用户状态类型及数量: ")  
所有用户状态类型及数量:
```

```
scala> spark.sql("""  
    | SELECT run_name, COUNT(*) as cnt  
    | FROM mediamatch_usermsg  
    | GROUP BY run_name  
    | ORDER BY cnt DESC  
    | """).show(20, false)
```

```
+-----+-----+  
|run_name|cnt  |  
+-----+-----+  
|正常    |59490|  
|主动暂停|15847|  
|主动销户|13879|  
|欠费暂停|9239 |  
|被动销户|1446 |  
|冲正    |67   |  
|创建    |30   |  
|销号    |2    |
```

```
scala> // 2. 统计四种有效状态的用户数
```

```
scala> println("四种有效状态用户数: ")  
四种有效状态用户数:
```

```
scala> spark.sql("""  
    | SELECT run_name, COUNT(*) as cnt  
    | FROM mediamatch_usermsg  
    | WHERE run_name IN ('正常', '欠费暂停', '主动暂停', '主动销户')  
    | GROUP BY run_name  
    | """).show()
```

```
+-----+-----+  
|run_name| cnt|  
+-----+-----+  
|欠费暂停| 9239|  
|主动暂停|15847|  
|主动销户|13879|  
|   正常|59490|  
+-----+-----+
```

（三）用户收视行为无效数据

用户收视行为信息表记录了用户的每次收视时长，对于时长过短以及过长的数据记录都是无效的，过短是因为用户切换频道时产生的记录，过长可能是因为用户没有关闭机顶盒造成的记录，但过短与过长的范围需要进行统计分析得到。


```

== Source file context for tree position ==

3
4 object $eval {
5   lazy val $result = res36
6   lazy val $print: _root_.java.lang.String = {
7     $iw
8
9   "" )

```

hour_bin	cnt	pct
0	4465601	93.92
1	280086	5.89
2	4865	0.10
3	2439	0.05
4	1451	0.03

记录观看时长小于一小时的各区间记录数及占比（以分钟划分）

```

2026-06-26 08:53:12,201 WARN window.WindowExec: No Partition Defined for Window operation! Moving all data to a single partition, this can cause serious performance degradation.

```

minute_bin	cnt	pct
0	849137	19.02
1	453136	10.15
2	282420	6.32
3	212649	4.76
4	162935	3.65
5	192035	4.30
6	116067	2.60
7	108055	2.42
8	100398	2.25
9	87188	1.95
10	97834	2.19
11	82628	1.85
12	66950	1.50
13	58667	1.31
14	56501	1.27
15	60516	1.36
16	48117	1.08
17	46965	1.05
18	46235	1.04
19	42832	0.96
20	70427	1.58
21	42650	0.96
22	46295	1.04
23	37574	0.84
24	38925	0.87
25	57161	1.28
26	37617	0.84
27	37529	0.84
28	36350	0.81
29	33657	0.75

only showing top 30 rows

得到观看时长小于 1 分钟的各区间记录数及占比（以秒级划分）

```

2026-06-26 08:54:51,622 WARN window.WindowExec: No Partition Defined for Window operation! Moving all data to a single partition, this can cause serious performance degradation.

```

second_bin	cnt	pct
0	19449	2.29
1	2793	0.33

2	2485	0.29
3	2450	0.29
4	2365	0.28
5	2599	0.31
6	2385	0.28
7	2533	0.30
8	2544	0.30
9	2593	0.31
10	2464	0.29
11	2476	0.29
12	2464	0.29
13	2396	0.28
14	2482	0.29
15	2487	0.29
16	2515	0.30
17	2801	0.33
18	2819	0.33
19	2938	0.35
20	24119	2.84
21	41343	4.87
22	38473	4.53
23	35561	4.19
24	32760	3.86
25	31450	3.70
26	29770	3.51
27	27802	3.27
28	27027	3.18
29	25778	3.04
30	23781	2.80
31	23177	2.73
32	22164	2.61
33	21728	2.56
34	20798	2.45

```
scala> val totalCount = spark.sql("SELECT COUNT(*) FROM media_index_temp").collect()(0).getLong(0)
totalCount: Long = 4754442

scala> val invalidPct = invalidCount * 100.0 / totalCount
invalidPct: Double = 18.52478166733341

scala> println(s"总记录数: $totalCount")
总记录数: 4754442

scala> println(s"无效记录数: $invalidCount")
无效记录数: 880750

scala> println(f"无效记录占比: $invalidPct%.2f%")
无效记录占比: 18.52%
```

2.3 标签阈值探索

本小节将对电视用户账单进行分析、对宽带用户的账单进行分析、对电视入网程度标签阈值进行探索、对宽带入网程度标签阈值进行探索，制定标签规则，为下一步骤的数据预处理提供处理依据。

（一）电视用户账单分析

通过对账单信息数据中的电视用户的账单信息进行分析，得到一个电视用户

的消费水平标签划分规则。

- 对电视用户的账单数据进行探索分析，统计每个用户的月均消费金额，然后对所有用户的月均消费金额作基本的统计分析。

```
scala> // 1. 电视用户月均消费基本统计（均值、标准差、最值）

scala> println("电视用户月均消费金额统计：")
电视用户月均消费金额统计：

scala> spark.sql("""
  SELECT
    AVG(avg_fee) AS mean,
    STDDEV(avg_fee) AS stddev,
    MIN(avg_fee) AS min,
    MAX(avg_fee) AS max
  FROM (
    SELECT
      phone_no,
      AVG(CAST(should_pay AS DOUBLE) - CAST(favour_fee AS DOUBLE)) AS avg_fee
    FROM mmconsume_billevents
    WHERE sm_name NOT LIKE '%珠江宽频%'
      AND should_pay RLIKE '^[0-9.]+$'
      AND favour_fee RLIKE '^[0-9.]+$'
    GROUP BY phone_no
  ) t
""").show()

+-----+-----+-----+-----+
|              mean|          stddev|    min|    max|
+-----+-----+-----+-----+
|19.456327507123696|22.739469718782882|-0.4|3780.0|
+-----+-----+-----+-----+
```

- 将每月消费金额按 10 的间隔划分，通过range_fee 分组。

```
scala> // 2. 按10元间隔分组统计

scala> println("电视用户月均消费分布（按10元间隔）：")
电视用户月均消费分布（按10元间隔）：

scala> spark.sql("""
  SELECT
    FLOOR(avg_fee/10)*10 AS fee_range,
    COUNT(*) AS cnt
  FROM (
    SELECT
      phone_no,
      AVG(CAST(should_pay AS DOUBLE) - CAST(favour_fee AS DOUBLE)) AS avg_fee
    FROM mmconsume_billevents
    WHERE sm_name NOT LIKE '%珠江宽频%'
      AND should_pay RLIKE '^[0-9.]+$'
      AND favour_fee RLIKE '^[0-9.]+$'
    GROUP BY phone_no
  ) t
  GROUP BY FLOOR(avg_fee/10)*10
  ORDER BY fee_range
""").show(20, false)
```

fee_range	cnt
-10	42
0	7381
10	6424
20	18850
30	105
40	20
50	4
60	14
70	3
80	3
90	3
110	1
180	2
210	1
310	1

- 电视用户月均消费金额大致呈正态分布，主要集中在0 至50 的区间范围内，其中占比最大的是大于等于20 而小于30 的消费区间，这是因为大部分电视用户每月都只缴纳26.5 元基本电视费用。筛选电视用户月均消费大于等于-10 且小于等于90元的用户数据，统计其平均值和标准差。

```
scala> // 3. 筛选 -10~90 元区间，计算均值和标准差

scala> println("电视用户月均消费 -10~90 元区间统计：")
电视用户月均消费 -10~90 元区间统计：

scala> spark.sql("""
  SELECT
    AVG(avg_fee) AS mean,
    STDDEV(avg_fee) AS stddev
  FROM (
    SELECT
      phone_no,
      AVG(CAST(should_pay AS DOUBLE) - CAST(favour_fee AS DOUBLE)) AS avg_fee
    FROM mmconsume_bill_events
    WHERE sm_name NOT LIKE '%珠江宽频%'
      AND should_pay RLIKE '^[0-9.]+$'
      AND favour_fee RLIKE '^[0-9.]+$'
    GROUP BY phone_no
  ) t
  WHERE avg_fee >= -10 AND avg_fee <= 90
""").show()
```

mean	stddev
19.296650884093943	8.554387220761347

（二）宽带用户的账单分析

本小节将通过对账单信息数据中的宽带用户的账单信息进行分析，得到一个

宽带用户的消费水平标签划分规则。

- 对宽带用户的账单数据进行探索分析，统计每个用户的月均消费金额，然后对所有用户的月均消费金额作基本的统计分析。

```
scala> println("宽带用户月均消费金额统计：")
宽带用户月均消费金额统计：

scala> spark.sql("""
  SELECT
    AVG(avg_fee) AS mean,
    STDDEV(avg_fee) AS stddev,
    MIN(avg_fee) AS min,
    MAX(avg_fee) AS max
  FROM (
    SELECT
      phone_no,
      AVG(CAST(should_pay AS DOUBLE) - CAST(favour_fee AS DOUBLE)) AS avg_fee
    FROM mmconsume_billevents
    WHERE sm_name LIKE '%珠江宽频%'
      AND should_pay RLIKE '^[0-9.]+$'
      AND favour_fee RLIKE '^[0-9.]+$'
    GROUP BY phone_no
  ) t
""").show()
```

mean	stddev	min	max
33.182467660073875	13.10651130544033	-0.5	180.0

- 将每月消费金额按 10 的间隔划分，通过range_fee 分组。

```
COUNT(*) AS cnt
FROM (
  SELECT
    phone_no,
    AVG(CAST(should_pay AS DOUBLE) - CAST(favour_fee AS DOUBLE)) AS avg_fee
  FROM mmconsume_billevents
  WHERE sm_name LIKE '%珠江宽频%'
    AND should_pay RLIKE '^[0-9.]+$'
    AND favour_fee RLIKE '^[0-9.]+$'
  GROUP BY phone_no
) t
GROUP BY FLOOR(avg_fee/10)*10
ORDER BY fee_range
""").show(20, false)
```

fee_range	cnt
-10	1
0	67
10	523
20	2484
30	3804
40	1923
50	585
60	209
70	29
80	13
90	6
100	4
110	1
120	5
150	1
180	1

- 宽带用户的消费金额集中在0 元至90 元之间，其中消费金额在 10 至20

区间内的用户最多，为了使宽带消费金额的均值和标准差更加稳定，过滤消费金额小于 0 或大于 90 的记录，再求其均值和标准差和中位数。

```
scala> // 3. 筛选 0~90 元区间，计算均值和标准差

scala> println("宽带用户月均消费 0~90 元区间统计：")
宽带用户月均消费 0~90 元区间统计：

scala> spark.sql("""
  SELECT
    AVG(avg_fee) AS mean,
    STDDEV(avg_fee) AS stddev
  FROM (
    SELECT
      phone_no,
      AVG(CAST(should_pay AS DOUBLE) - CAST(favour_fee AS DOUBLE)) AS avg_fee
    FROM mmconsume_billevents
    WHERE sm_name LIKE '%珠江宽频%'
      AND should_pay RLIKE '^[0-9.]+$'
      AND favour_fee RLIKE '^[0-9.]+$'
    GROUP BY phone_no
  ) t
  WHERE avg_fee >= 0 AND avg_fee <= 90
""").show()
+-----+-----+
|          mean|          stddev|
+-----+-----+
|33.03814130182351|12.625229379105043|
+-----+-----+
```

（三）电视入网程度标签阈值探索

通过对用户信息数据中的电视用户的开户时间信息进行分析，得到一个电视用户的入网程度标签划分规则。

- 把用户基本信息表的 `open_time` 字段的值与当前时间作差并把差值转为以年为单位，然后统计所有用户中入网时长的最值、均值。
- 通过 `years` 分组统计每个入网户时长值的用户数。

```
scala> println("=== 电视用户入网时长统计===")
=== 电视用户入网时长统计===

scala> spark.sql("""
  SELECT
    AVG(tenure_years) AS mean,
    STDDEV(tenure_years) AS stddev,
    MIN(tenure_years) AS min,
    MAX(tenure_years) AS max
  FROM (
    SELECT
      DATEDIFF(CURRENT_DATE(), TO_DATE(SUBSTR(open_time, 1, 10), 'yyyy-MM-dd')) / 365.0 AS tenure_years
    FROM mediamatch_usermsg
    WHERE sm_name IN ('数字电视', '互动电视', '甜果电视')
      AND open_time IS NOT NULL
      AND LENGTH(open_time) >= 10
    ) t
""").show()
```

mean	stddev	min	max
18.6041455038	1.203990773445615	12.202740	21.767123

```
scala> println("=== 电视用户入网时长分布（按年分组）===")
=== 电视用户入网时长分布（按年分组）===

scala> spark.sql("""
  SELECT
    FLOOR(tenure_years) AS tenure_year,
    COUNT(*) AS cnt
  FROM (
    SELECT
      DATEDIFF(CURRENT_DATE(), TO_DATE(SUBSTR(open_time, 1, 10), 'yyyy-MM-dd')) / 365.0
    AS tenure_years
    FROM mediamatch_usermsg
    WHERE sm_name IN ('数字电视', '互动电视', '甜果电视')
      AND open_time IS NOT NULL
      AND LENGTH(open_time) >= 10
    ) t
  GROUP BY FLOOR(tenure_years)
  ORDER BY tenure_year
""").show(30, false)
```

tenure_year	cnt
12	1498
13	3
17	2900
18	18060
19	21016
20	64
21	5

（四）宽带入网程度标签阈值探索

在对宽带用户的入网时长统计分析中，主要统计分析宽带用户中入网时长的最值、均值、标准差、中位数及其分布情况。

```
scala> println("=== 宽带用户入网时长统计===")
=== 宽带用户入网时长统计===

scala> spark.sql("""
  SELECT
    AVG(tenure_years) AS mean,
    STDDEV(tenure_years) AS stddev,
    MIN(tenure_years) AS min,
    MAX(tenure_years) AS max
  FROM (
    SELECT
      DATEDIFF(CURRENT_DATE(), TO_DATE(SUBSTR(open_time, 1, 10), 'yyyy-MM-dd')) / 365.0
    AS tenure_years
    FROM mediamatch_usermsg
    WHERE sm_name LIKE '%珠江宽频%'
      AND force = '宽带生效'
      AND sm_code = 'b0'
      AND open_time IS NOT NULL
      AND LENGTH(open_time) >= 10
    ) t
  """).show()
+-----+-----+-----+-----+
|      mean|      stddev|      min|      max|
+-----+-----+-----+-----+
|17.4700528226|3.378670749687299|12.178082|24.128767|
+-----+-----+-----+-----+
```

通过years 分组统计每个入网户时长值的用户数。

```
scala> println("=== 宽带用户入网时长分布（按年分组）===")
=== 宽带用户入网时长分布（按年分组）===

scala> spark.sql("""
  SELECT
    FLOOR(tenure_years) AS tenure_year,
    COUNT(*) AS cnt
  FROM (
    SELECT
      DATEDIFF(CURRENT_DATE(), TO_DATE(SUBSTR(open_time, 1, 10), 'yyyy-MM-dd')) / 365.0
    AS tenure_years
    FROM mediamatch_usermsg
    WHERE sm_name LIKE '%珠江宽频%'
      AND force = '宽带生效'
      AND sm_code = 'b0'
      AND open_time IS NOT NULL
      AND LENGTH(open_time) >= 10
    ) t
  GROUP BY FLOOR(tenure_years)
  ORDER BY tenure_year
  """).show(30, false)
+-----+-----+
|tenure_year|cnt|
+-----+-----+
|12|477|
|17|156|
|18|257|
|19|311|
|20|248|
|21|91|
|22|44|
|23|10|
|24|1|
+-----+-----+
```

2.4 数据预处理

编写Spark程序（Scala/Java/Python均可）对5张原始表进行数据清洗。数据清洗的规则如表2-5所示。进行相应的数据预处理。将清洗后的数据写回Hive新表，并统计清洗前后记录数变化，验证清洗规则的执行效果。

```
spark-sql> select 'usermsg' as table_name,
>               (select count(*) from mediamatch_usermsg) as original,
>               (select count(*) from mediamatch_usermsg_clean) as cleaned
> union all
> select 'userevent',
>       (select count(*) from mediamatch_userevent),
>       (select count(*) from mediamatch_userevent_clean)
> union all
> select 'billevents',
>       (select count(*) from mmconsume_billevents),
>       (select count(*) from mmconsume_billevents_clean)
> union all
> select 'order_index',
>       (select count(*) from order_index),
>       (select count(*) from order_index_clean)
> union all
> select 'media_index',
>       (select count(*) from media_index),
>       (select count(*) from media_index_clean);
```

```
usermsg 100000 44757
userevent 3000 2910
billevents 439158 409418
order_index 608514 517166
media_index 4754442 113278
Time taken: 82.713 seconds, Fetched 5 row(s)
```

代码展示：

1. 清洗用户信息表

```
drop table if exists mediamatch_usermsg_clean;
```

```
create table mediamatch_usermsg_clean as
```

```
with temp as (
```

```
    select *,
```

```
        row_number() over (partition by phone_no order by run_time desc) as rn
```

```
    from mediamatch_usermsg
```

```
where owner_name not rlike '^E[ABCDE]级'

and owner_code not in ('02','09','10')

and sm_name in ('珠江宽频','数字电视','互动电视','甜果电视')

and run_name in ('正常','主动暂停','欠费暂停','主动销户')

)

select * from temp where rn = 1;
```

2. 清洗用户状态变更表

```
drop table if exists mediamatch_userevent_clean;

create table mediamatch_userevent_clean as

select distinct *

from mediamatch_userevent

where run_name in ('正常','主动暂停','欠费暂停','主动销户');
```

3. 清洗账单表

```
drop table if exists mmconsume_billevents_clean;

create table mmconsume_billevents_clean as

select distinct *

from mmconsume_billevents

where owner_name not rlike '^E[ABCDE]级'

and owner_code not in ('02','09','10')

and sm_name in ('珠江宽频','数字电视','互动电视','甜果电视');
```

4. 清洗订单表

```
drop table if exists order_index_clean;

create table order_index_clean as

select distinct *

from order_index

where owner_name not rlike '^E[ABCDE]级'

and owner_code not in ('02','09','10')
```



```
and sm_name in ('珠江宽频','数字电视','互动电视','甜果电视')
and run_name in ('正常','主动暂停','欠费暂停','主动销户');
```

5. 清洗收视行为表（先建有效用户临时表）

```
drop table if exists media_index_clean;
create table media_index_clean as
select m.*
from media_index m
inner join (select distinct phone_no from mediamatch_usermsg_clean) v
on m.phone_no = v.phone_no
where m.owner_name not rlike '^E[ABCDE]级'
and m.owner_code not in ('02','09','10')
and m.sm_name in ('珠江宽频','数字电视','互动电视','甜果电视')
and cast(m.duration as double) >= 20000
and cast(m.duration as double) <= 18000000
and not (
    m.res_type = '0'
    and regexp_extract(m.origin_time, ':[0-9]{2})$', 1) = '00'
    and regexp_extract(m.end_time, ':[0-9]{2})$', 1) = '00'
);
```

```
select 'usermsg' as 表名,
       (select count(*) from mediamatch_usermsg) as 原始行数,
       (select count(*) from mediamatch_usermsg_clean) as 清洗后行数,
       concat(cast((1 - (select count(*) from mediamatch_usermsg_clean) / (select count(*)
from mediamatch_usermsg)) * 100 as decimal(5,2)), '%') as 剔除比例
union all
select 'userevent',
       (select count(*) from mediamatch_userevent),
       (select count(*) from mediamatch_userevent_clean),
```

```

concat(cast((1 - (select count(*) from mediamatch_userevent_clean) / (select count(*)
from mediamatch_userevent)) * 100 as decimal(5,2)), '%')
union all
select 'billevents',
      (select count(*) from mmconsume_billevents),
      (select count(*) from mmconsume_billevents_clean),
      concat(cast((1 - (select count(*) from mmconsume_billevents_clean) / (select count(*)
from mmconsume_billevents)) * 100 as decimal(5,2)), '%')
union all
select 'order_index',
      (select count(*) from order_index),
      (select count(*) from order_index_clean),
      concat(cast((1 - (select count(*) from order_index_clean) / (select count(*) from
order_index)) * 100 as decimal(5,2)), '%')
union all
select 'media_index',
      (select count(*) from media_index),
      (select count(*) from media_index_clean),
      concat(cast((1 - (select count(*) from media_index_clean) / (select count(*) from
media_index)) * 100 as decimal(5,2)), '%');

```

三、SVM 模型构建与用户画像生成

3.1 构建特征列和标签列数据

在预处理后的数据中没有算法模型可识别出来的特征列和标签列，所以为了后续模型的构建，此小节需要对预处理后的数据进行构建特征列和标签列。

（一）根据用户月均消费金额，每个用户的入网时长，每个用户平均每天看多久电视构建特征列，通过 join 将三者进行连接。

(二)根据 mediamatch_usermsg 选出 run_name 字段为主动暂停或主动销户的用户并贴上类别 0, 0 代表用户为不挽留用户; 根据 mediamatch_usermsg 选出 run_name 字段为正常并且是活跃的用户贴上类别 1, 1 代表用户为挽留用户。

(三) order_index 数据过滤 run_name 等于“正常”; offername 过滤掉以下关键字“废”“赠送”“免费体验”“提速”“提价”“转网优惠”“测试”“虚拟”“空包”。

通过得到了一份 DataFrame 的数据 unionData, 分别为用户 ID、电视消费水平、电视入网时长、电视依赖度、类别等。后续可利用这份数据来进行 SVM 建模。

关键代码展示:

1. 月均消费

```
df_spend = spark.table("mmconsume_billevents_clean") \
    .withColumn("net_pay", col("should_pay").cast("double") - col("favour_fee").cast("double")) \
    .groupBy("phone_no") \
    .agg(avg("net_pay").alias("avg_spend"))
```

2. 入网时长

```
df_tenure = spark.table("mediamatch_usermsg_clean") \
    .withColumn("tenure", datediff(current_date(), col("open_time").cast("timestamp"))) /
365.25)
```

3. 日均观看时长

```
df_watch = spark.table("media_index_clean") \
    .groupBy("phone_no") \
    .agg((sum(col("duration").cast("double"))) / 3600000 / 365.25).alias("avg_daily_hours")) \
    .withColumn("avg_daily_hours", coalesce(col("avg_daily_hours"), lit(0.0)))
```

合并特征, 并将所有可能为空的列填充为0

```
df_features = df_tenure.select("phone_no", "tenure") \
    .join(df_spend, on="phone_no", how="left") \
```

```

.join(df_watch, on="phone_no", how="left") \

.fillna({"avg_spend": 0.0, "avg_daily_hours": 0.0, "tenure": 0.0})

print("特征数据行数（用户数）：{}".format(df_features.count()))

# 标签列

df_status = spark.table("mediamatch_usermsg_clean").select("phone_no", "run_name")

df_labeled = df_features.join(df_status, on="phone_no", how="inner") \

    .withColumn("label",

                when((col("run_name").isin("主动暂停", "主动销户")), 0.0)

                .when((col("run_name") == "正常") & (col("avg_daily_hours") > 0), 1.0)

                .otherwise(-1.0)

    ) \

    .filter(col("label") >= 0)

print("可训练样本数：{}".format(df_labeled.count()))

df_labeled.groupBy("label").count().show()

# 特征组装、标准化、SVM

feature_cols = ["avg_spend", "tenure", "avg_daily_hours"]

assembler = VectorAssembler(inputCols=feature_cols, outputCol="features_raw",
                             handleInvalid="skip")

scaler = StandardScaler(inputCol="features_raw", outputCol="features", withStd=True,
                         withMean=True)

svm = LinearSVC(labelCol="label", featuresCol="features", maxIter=50, regParam=0.01)

pipeline = Pipeline(stages=[assembler, scaler, svm])

train_data, test_data = df_labeled.randomSplit([0.8, 0.2], seed=42)

model = pipeline.fit(train_data)

predictions = model.transform(test_data)

accuracy = predictions.filter(predictions.label == predictions.prediction).count() /
predictions.count()

```

```

print("准确率 (Accuracy): {:.4f}".format(accuracy))

evaluator_roc = BinaryClassificationEvaluator(labelCol="label",
rawPredictionCol="rawPrediction", metricName="areaUnderROC")

auroc = evaluator_roc.evaluate(predictions)

print("AUROC: {:.4f}".format(auroc))

evaluator_pr = BinaryClassificationEvaluator(labelCol="label",
rawPredictionCol="rawPrediction", metricName="areaUnderPR")

auprc = evaluator_pr.evaluate(predictions)

print("AUPRC: {:.4f}".format(auprc))

# 预测所有用户并保存

df_all_features = df_features.join(df_status, on="phone_no", how="inner")

df_all_pred = model.transform(df_all_features)

df_result = df_all_pred.select("phone_no", col("prediction").alias("label"))

df_result.write.mode("overwrite").saveAsTable("svm_prediction")

```

3.2 SVM 预测用户是否挽留

在构建模型之前，还需将特征列数据转化成 `RDD[LabelPoint]` 类型，消除量纲影响，对数据做标准化处理，然后对数据集划分，再来构建 SVM 模型，对模型效果进行评价，最后使用构建好的模型预测用户是否挽留。

对特征列数据做标准化处理，为了后面对模型进行评估，根据二八原则将标准化数据划分成训练集和验证集，用训练集来建立模型，验证集用来评价模型。

使用验证集来验证模型的效果，计算模型的准确率、AUROC 值和 AUPRC 值。

```

任务 3.1 & 3.2: 特征构建与 SVM 模型训练
=====
特征数据行数（用户数）：44757
可训练样本数：13612
+-----+-----+
|label|count|
+-----+-----+
|  0.0|13458|
|  1.0|  154|
+-----+-----+

```



```
准确率 (Accuracy): 0.9981
AUROC: 0.9996
AUPRC: 0.9510
```

```
预测结果已写入，总用户数：44757
```

```
=====
任务 3.1 & 3.2 完成!
```

3.3 构建用户画像

通过前几节得到的结论，来实现消费内容、电视消费水平、宽带消费水平、宽带产品带宽、销售品名称、业务品牌、电视入网程度和宽带入网程度的用户画像。

基于清洗后的数据、模型预测结果以及前期制定的阈值规则，为每个用户生成以下8个维度的标签，并存储为宽表或分区表：

（一）消费内容

选择 phone_no、fee_code 字段并去重；根据 fee_code 字段来贴标签，标签规则如下：

- （1）fee_code=0J 或 0B 或 0Y 则标签为直播；
- （2）fee_code=0X 则标签为应用；
- （3）fee_code=0T 则标签为付费频道；
- （4）fee_code=0W 或 0L 或 0Z 或 0K 则标签为宽带；
- （5）fee_code=0D 则标签为点播，fee_code=0H 则标签为回看；
- （6）fee_code=0U 则标签为有线电视收视费。

1. 消费内容标签（从账单表）

```
df_content = spark.table("mmconsume_billevents_clean") \
    .select("phone_no", "fee_code") \
    .distinct() \
    .withColumn("content_label",
                when(col("fee_code").isin("0J", "0B", "0Y"), "直播")
                .when(col("fee_code") == "0X", "应用")
                .when(col("fee_code") == "0T", "付费频道")
```

```

        .when(col("fee_code").isin("0W", "0L", "0Z", "0K"), "
宽带")

        .when(col("fee_code") == "0D", "点播")

        .when(col("fee_code") == "0H", "回看")

        .when(col("fee_code") == "0U", "有线电视收视费")

        .otherwise("其他")

    )

```

每个用户取第一个消费内容（如果有多个，取第一个）

```

window_content = Window.partitionBy("phone_no").orderBy("fee_code")

df_content_final = df_content.withColumn("rn",
row_number().over(window_content)) \

    .filter(col("rn") == 1) \

    .select("phone_no", "content_label")

```

（二）电视消费水平

根据 sm_name 字段不包含珠江宽频筛选出电视的数据；计算个月数据中 should_pay - favour_fee 的月平均值 X，标签规则如下：

- （1）如果 $-26.3 < X < 26.3$ 则贴上电视超低消费标签；
- （2）如果 $26.3 \leq X < 26.3 + 20$ 则贴上电视低消费标签；
- （3）如果 $26.3 + 20 \leq X < 26.3 + 40$ 则贴上电视中等消费标签；
- （4）如果 $X \geq 26.3 + 40$ 则贴上电视高消费标签。

2. 电视消费水平（从账单表，不含宽带）

```

df_tv_spend = spark.table("mmconsume_billevents_clean") \

    .filter(~col("sm_name").rlike("珠江宽频")) \

    .withColumn("net_pay", col("should_pay").cast("double") -
col("favour_fee").cast("double")) \

```

```

    .groupBy("phone_no") \
    .agg(avg("net_pay").alias("avg_tv_spend"))

df_tv_level = df_tv_spend.withColumn("tv_spend_label",
    when(col("avg_tv_spend") < 26.3, "电视超低消费")
    .when((col("avg_tv_spend") >= 26.3) & (col("avg_tv_spend") < 46.3),
"电视低消费")
    .when((col("avg_tv_spend") >= 46.3) & (col("avg_tv_spend") < 66.3),
"电视中等消费")
    .when(col("avg_tv_spend") >= 66.3, "电视高消费")
    .otherwise("未知"))
).select("phone_no", "tv_spend_label")

```

（三）宽带消费水平

根据 sm_name 包含珠江宽频筛选宽带的用户数据，计算 3 个月数据 should_pay - favour_fee 的月平均值 Y，标签规则如下：

- （1）如果 $Y \leq 29$ 则贴上宽带低消费标签；
- （2）如果 $29 < Y \leq 48$ 则贴上宽带中消费标签；
- （3）如果 $Y > 48$ 则贴上宽带高消费标签。

3. 宽带消费水平（从账单表，仅宽带）

```

df_bb_spend = spark.table("mmconsume_billevents_clean") \
    .filter(col("sm_name").rlike("珠江宽频")) \
    .withColumn("net_pay", col("should_pay").cast("double") -
col("favour_fee").cast("double")) \
    .groupBy("phone_no") \
    .agg(avg("net_pay").alias("avg_bb_spend"))

```

```
df_bb_level = df_bb_spend.withColumn("bb_spend_label",
    when(col("avg_bb_spend") <= 29, "宽带低消费")
    .when((col("avg_bb_spend") > 29) & (col("avg_bb_spend") <= 48), "
宽带中消费")
    .when(col("avg_bb_spend") > 48, "宽带高消费")
    .otherwise("无宽带消费")
).select("phone_no", "bb_spend_label")
```

（四）带宽销售品名称

过滤 cost 小于等于 0 的数据，且 offername 不包含空包的记录；根据 sm_name 区分电视和宽带，sm_name = '珠江宽频' 的为宽带，sm_name 不包含珠江宽频的为电视，标签规则如下：

- （1）电视主销售品：找出 mode_time='Y'，offertype=0，prodstatus='YY' 且 effdate ≤ 当前时间 ≤ expdate 且 optdate 最大的数据；
- （2）电视附属销售品：找出 mode_time='Y'，offertype=0，prodstatus='YY' 且 effdate ≤ 当前时间 ≤ expdate 的数据；
- （3）宽带：找出 effdate ≤ 当前时间 ≤ expdate 且 optdate 最大的数据。

筛选出来的数据选择 phone_no、offername 字段并去重，然后根据 offername 字段贴标签。

4. 带宽销售品名称（从订单表）

过滤有效订单

```
df_order_valid = spark.table("order_index_clean") \
    .filter(col("cost").cast("double") > 0) \
    .filter(col("offername").isNotNull() & (col("offername") != "空包")) \
    .withColumn("effdate", col("effdate").cast("timestamp")) \
    .withColumn("expdate", col("expdate").cast("timestamp")) \
    .withColumn("optdate", col("optdate").cast("timestamp")) \
    .withColumn("current_ts", current_date().cast("timestamp"))
```

```

# 筛选有效期内且 optdate 最大的记录

window_order =
Window.partitionBy("phone_no").orderBy(col("optdate").desc())

df_order_best = df_order_valid \

    .filter((col("effdate") <= col("current_ts")) & (col("expdate") >=
col("current_ts")))) \

    .filter((col("mode_time") == "Y") & (col("offertype") == "0") &
(col("prodstatus") == "YY")) \

    .withColumn("rn", row_number().over(window_order)) \

    .filter(col("rn") == 1) \

    .select("phone_no", col("offername").alias("product_label"))

```

（五）业务品牌

选择 phone_no、sm_name 字段并去重，删除 sm_name = '模拟有线电视' 或 '番通' 的数据，根据 sm_name 字段来贴标签，标签规则如下：

- （1）若 sm_name = '互动电视' 则标签为互动电视；
- （2）若 sm_name = '数字电视' 则标签为数字电视；
- （3）若 sm_name = '甜果电视' 则标签为甜果电视；
- （4）若 sm_name = '珠江宽频' 则标签为珠江宽频。

5. 业务品牌（从用户表）

```

df_brand = spark.table("mediamatch_usermsg_clean") \

    .select("phone_no", "sm_name") \

    .distinct() \

    .filter(~col("sm_name").isin("模拟有线电视", "番通")) \

    .withColumn("brand_label",

```

```

        when(col("sm_name") == "互动电视", "互动电视")
        .when(col("sm_name") == "数字电视", "数字电视")
        .when(col("sm_name") == "甜果电视", "甜果电视")
        .when(col("sm_name") == "珠江宽频", "珠江宽频")
        .otherwise("其他")
    ) \
    .select("phone_no", "brand_label")

```

（六）电视入网程度

筛选 sm_name 包含“互动”“甜果”“数字”的记录。根据 phone_no 字段来分组，找出 open_time（用户开户时间）字段与当前时间作比差得到 T（单位：年），标签规则如下：

- （1）若 $T > 13$ 则标签为老用户；
- （2）若 $T \leq 9$ 则标签为新用户；
- （3）若 $9 < T \leq 13$ 则标签为中等用户。

6. 电视入网程度（从用户表，不含宽带）

```

df_tv_tenure = spark.table("mediamatch_usermsg_clean") \
    .filter(~col("sm_name").rlike("珠江宽频")) \
    .withColumn("tenure", datediff(current_date(),
col("open_time").cast("timestamp")) / 365.25) \
    .withColumn("tv_tenure_label",
        when(col("tenure") <= 9, "新用户")
        .when((col("tenure") > 9) & (col("tenure") <= 13), "
中等用户")
        .when(col("tenure") > 13, "老用户")
        .otherwise("未知")
    ) \

```



```
.select("phone_no", "tv_tenure_label")
```

（七）宽带入网程度

选择 sm_name 包含“珠江宽频”且 force 等于“宽带生效”且 sm_code = b0 的数据。

根据 phone_no 字段来分组，找出 open_time 字段（用户的开户时间）与当前时间作比差得到 T，标签规则如下：

- （1）若 $T > 14$ 则标签为老用户；
- （2）若 $T \leq 8$ 则标签为新用户；
- （3）若 $8 < T \leq 14$ 则标签为中等用户。

7. 宽带入网程度（从用户表，仅宽带）

```
df_bb_tenure = spark.table("mediamatch_usermsg_clean") \
    .filter(col("sm_name").rlike("珠江宽频")) \
    .filter(col("force") == "宽带生效") \
    .filter(col("sm_code") == "b0") \
    .withColumn("tenure", datediff(current_date(),
col("open_time").cast("timestamp")) / 365.25) \
    .withColumn("bb_tenure_label",
        when(col("tenure") <= 8, "新用户")
        .when((col("tenure") > 8) & (col("tenure") <= 14), "
中等用户")
        .when(col("tenure") > 14, "老用户")
        .otherwise("未知")
    ) \
    .select("phone_no", "bb_tenure_label")
```

（八）用户是否挽留

使用了 SVM 算法预测用户的挽留状态，并将预测结果保存在 Hive 中 user_profile 库下的 svm_prediction，因此用户是否挽留标签可根据 svm_prediction 表的 label 字段进行计算，label 等于 1 的为挽留用户，label 等于 0 为不挽留用户。

8. 用户是否挽留（从 SVM 预测结果）

```
df_svm = spark.table("svm_prediction") \
    .withColumn("retain_label",
                when(col("label") == 1.0, "挽留用户")
                .when(col("label") == 0.0, "不挽留用户")
                .otherwise("未知")
    ) \
    .select("phone_no", "retain_label")
```

9. 合并所有标签为宽表

从用户表获取所有有效用户

```
df_users =
spark.table("mediamatch_usermsg_clean").select("phone_no").distinct()
```

逐步左连接

```
df_profile = df_users \
    .join(df_content_final, on="phone_no", how="left") \
    .join(df_tv_level, on="phone_no", how="left") \
    .join(df_bb_level, on="phone_no", how="left") \
    .join(df_brand, on="phone_no", how="left") \
    .join(df_tv_tenure, on="phone_no", how="left") \
    .join(df_bb_tenure, on="phone_no", how="left") \
    .join(df_svm, on="phone_no", how="left") \
    .join(df_order_best, on="phone_no", how="left") \
```

```
.fillna({
    "content_label": "无消费",
    "tv_spend_label": "无电视消费",
    "bb_spend_label": "无宽带消费",
    "brand_label": "未知品牌",
    "tv_tenure_label": "未知",
    "bb_tenure_label": "无宽带",
    "retain_label": "未知",
    "product_label": "无有效产品"
})
```

10. 写入 Hive 表

```
df_profile.write.mode("overwrite").saveAsTable("user_profile_final")
```

用户画像宽表展示

☑ 用户画像已保存到 user_profile.user_profile_final

🖼 画像样例（前10行）：

```
2026-06-24 00:10:58,578 WARN pool.HikariPool: HikariPool-1 - Thread starvation or clock leap detected (housekeeper delta=3m11s150ms807µs101ns).
2026-06-24 00:10:58,576 WARN pool.HikariPool: HikariPool-2 - Thread starvation or clock leap detected (housekeeper delta=3m11s138ms368µs976ns).
2026-06-24 00:11:02,204 WARN spark.HeartbeatReceiver: Removing executor driver with no recent heartbeats: 188956 ms exceeds timeout 120000 ms
2026-06-24 00:11:06,350 WARN spark.SparkContext: Killing executors is not supported by current scheduler.
```

phone_no	content_label	tv_spend_label	bb_spend_label	brand_label	tv_tenure_label	bb_tenure_label	retain_label	product_label
2000565	无消费	无电视消费	无宽带消费	互动电视	中等用户	无宽带	不挽留用户	无有效产品
2001020	无消费	无电视消费	无宽带消费	互动电视	未知	无宽带	不挽留用户	无有效产品
2001107	无消费	无电视消费	无宽带消费	珠江宽频	未知	中等用户	不挽留用户	无有效产品
2005311	无消费	无电视消费	无宽带消费	互动电视	中等用户	无宽带	不挽留用户	无有效产品
2006847	无消费	无电视消费	无宽带消费	互动电视	中等用户	无宽带	不挽留用户	无有效产品
2010800	无消费	无电视消费	无宽带消费	珠江宽频	未知	中等用户	不挽留用户	无有效产品
2012257	无消费	无电视消费	无宽带消费	数字电视	中等用户	无宽带	不挽留用户	无有效产品
2017494	无消费	无电视消费	无宽带消费	数字电视	老用户	无宽带	不挽留用户	无有效产品
2018704	无消费	无电视消费	无宽带消费	互动电视	老用户	无宽带	不挽留用户	无有效产品
2019426	无消费	无电视消费	无宽带消费	甜果电视	老用户	无宽带	不挽留用户	无有效产品

only showing top 10 rows

四、设计体会

4.1 理解与收获

- 通过本次课程考核，我完整走通了从环境搭建、数据导入、数据清洗、特征工程、阈值探索到用户画像输出的全流程。

2. Hadoop提供分布式存储，Hive构建数据仓库，Spark完成分布式计算，三者各司其职、协同工作，构成了大数据生态的核心闭环。这让我深刻认识到，大数据不是单点技术，而是一整套生态体系的组合应用。
3. 模型评估不能只看准确率在样本极度不平衡的情况下，准确率不一定是准确的。真正决定模型是否有用的是AUROC和AUPRC

4.2 遇到的主要问题及解决方法

问题	原因	解决方法
Dead datanodes	slave1/slave2 DataNode未启动	手动启动DataNode，修复HDFS
csv文件上传不成功	数据内存太大	用Windows的scp命令上传
Hive SQL执行卡住	YARN资源不足	改用HDFS命令直接统计
清洗脚本被系统 Killed	media_index数据量太大	改用流式处理，逐行读取
Python3未安装	系统默认Python2.7	使用Miniconda安装Python3
spark-shell报语法错误	系统Python2.7不支持Spark3.x语法	改用Scala脚本执行
yum源失效	CentOS默认源无法访问	更换阿里云镜像源

4.3 改进方面

1. 提前规划版本兼容性：先确认Hadoop/Hive/Spark/Python的版本兼容性再搭建环境，避免“一切就绪却发现关键组件用不了”的被动局面。

2. **尽早导入真实数据进行测试：**在每个组件安装完成后就导入小批量数据进行验证，及时发现问题，避免“最后一刻全部推倒重来”。
3. **掌握多种备用方案：**Hive执行SQL因YARN资源不足频繁失败时，我及时改用HDFS命令直接统计，绕过MapReduce的调度等待，大幅提升了效率。这让我认识到，面对技术障碍时不应固守单一方案。
4. **注重代码与文档的规范化：**实验过程中需要反复调整命令和脚本，清晰的注释和版本记录能够帮助快速定位问题、避免重复劳动。